SQL SINAV REHBERİ — DDL & DML KOMUTLARI

Vize Hazırlık Dokümanı

SENARYO: Üniversite Bilgi Sistemi

Bu rehberdeki TÜM örnekler aşağıdaki senaryo üzerine kuruludur. Böylece kafanda tutarlı bir tablo oluşacak.

Senaryomuz: Bir üniversitenin öğrenci, ders, not, öğretim görevlisi ve bölüm bilgilerini tutan bir veritabanı.

BÖLÜM 1: DDL (Data Definition Language) — Veri Tanımlama Dili

DDL, veritabanının *yapısını* (iskeletini) oluşturmak, değiştirmek ve silmek için kullanılır. DDL komutları veriyle değil, *şemayla* ilgilenir.

Temel DDL komutları: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`

1.1 CREATE DATABASE — Veritabanı Oluşturma

Mantık: Herhangi bir tablo oluşturmadan önce, o tabloların yaşayacağı bir veritabanı (database) lazım. Bunu bir bina gibi düşün — önce binayı yaparsın, sonra odaları (tabloları) döşersin.

```
-- En basit hali: Sadece isim vererek veritabanı oluştur
CREATE DATABASE UniversiteDB;


-- Veritabanını kullanmaya başlamak için USE komutu gerekir
-- Hangi veritabanında çalışacağımızı belirtiyoruz
USE UniversiteDB;
```

```
-- Eğer zaten varsa hata almamak için IF NOT EXISTS kullanılır
-- Aynı isimde veritabanı varsa komut sessizce geçilir, hata fırlatmaz
CREATE DATABASE IF NOT EXISTS UniversiteDB;
```

1.2 DROP DATABASE — Veritabanı Silme

Mantık: Veritabanını tamamen siler. İçindeki TÜM tablolar, veriler, indexler — her şey gider. Geri dönüşü yoktur.

```
-- Veritabanını tamamen sil
-- DİKKAT: Bu komut geri alınamaz!
DROP DATABASE UniversiteDB;


-- Eğer yoksa hata almamak için IF EXISTS kullanılır
-- Var olmayan bir veritabanını silmeye çalışırsan hata vermez
DROP DATABASE IF EXISTS UniversiteDB;
```

1.3 CREATE TABLE — Tablo Oluşturma

Mantık: Veritabanının içine veri saklayacağımız tabloları oluşturuyoruz. Her tablonun sütunları (kolonları) var ve her sütunun bir veri tipi var. Bu, uygulamadaki model sınıfına (class) benzer — hangi alanlar olacak, tipleri ne olacak, bunları burada tanımlıyoruz.

Temel Veri Tipleri (Sınavda bilmen gerekenler):

Tip	Açıklama	Örnek
INT	Tam sayı	1, 42, -7
VARCHAR(n)	Değişken uzunluklu metin (max n karakter)	'Emin', 'Gümüşhane'
CHAR(n)	Sabit uzunluklu metin (her zaman n karakter yer kaplar)	'TR', 'M'
DATE	Tarih (YYYY-MM-DD)	'2025-03-25'
DECIMAL(p,s)	Ondalıklı sayı (p: toplam basamak, s: ondalık basamak)	3.75, 98.50

FLOAT	**Yaklaşık ondalıklı sayı**	**3.14159**
TEXT	**Çok uzun metin**	**Paragraf uzunluğunda açıklamalar**
BOOLEAN	**Doğru/Yanlış**	**TRUE, FALSE**

Örnek 1: Basit tablo oluşturma

```
-- Bölümler tablosu: Her bölümün bir ID'si, adı ve kurulma tarihi var
CREATE TABLE bolumler (
    bolum_id INT,
    bolum_adi VARCHAR(100),
    kurulus_tarihi DATE
);
```

Örnek 2: Constraint'lerle (kısıtlamalarla) tablo oluşturma

```
-- Bölümler tablosu — bu sefer kısıtlamalarla birlikte
-- NOT NULL: Bu alan boş bırakılamaz
-- PRIMARY KEY: Tablodaki her satırı benzersiz şekilde tanımlayan anahtar
-- UNIQUE: Bu sütundaki değerler tekrar edemez (ama NULL olabilir)
CREATE TABLE bolumler (
    bolum_id INT PRIMARY KEY,              -- Her bölümün benzersiz kimliği
    bolum_adi VARCHAR(100) NOT NULL UNIQUE, -- Bölüm adı boş olamaz ve tekrar ede
    fakulte VARCHAR(100) NOT NULL,          -- Hangi fakülteye bağlı
    kurulus_tarihi DATE,                    -- NULL olabilir, zorunlu değil
    kontenjan INT DEFAULT 60               -- Belirtilmezse otomatik 60 olur
);
```

Örnek 3: AUTO_INCREMENT ile tablo oluşturma

```
-- Öğretim görevlileri tablosu
-- AUTO_INCREMENT: Her yeni kayıtta ID otomatik 1 artar
-- Biz ID vermeyiz, sistem kendi atar (1, 2, 3, ...)
CREATE TABLE ogretim_gorevlileri (
    ogretmen_id INT PRIMARY KEY AUTO_INCREMENT,
    ad VARCHAR(50) NOT NULL,
    soyad VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,             -- Her hocanın emaili farklı olmalı
    unvan VARCHAR(20) DEFAULT 'Öğr. Gör.', -- Belirtilmezse varsayılan unvan atan
    bolum_id INT NOT NULL                  -- Hangi bölümde çalışıyor
);
```

Örnek 4: FOREIGN KEY ile ilişkisel tablo oluşturma

```
-- Öğrenciler tablosu
-- FOREIGN KEY: Başka bir tablodaki PRIMARY KEY'e referans verir
-- Bu sayede tablolar arasında ilişki kurulur
CREATE TABLE ogrenciler (
    ogrenci_id INT PRIMARY KEY AUTO_INCREMENT,
    ad VARCHAR(50) NOT NULL,
    soyad VARCHAR(50) NOT NULL,
    dogum_tarihi DATE,
    cinsiyet CHAR(1),                       -- 'E' veya 'K' — sabit 1 karakter
    email VARCHAR(100) UNIQUE,
    kayit_tarihi DATE DEFAULT (CURRENT_DATE), -- Bugünün tarihi otomatik atanır
    bolum_id INT,
    -- FOREIGN KEY tanımı: bolum_id, bolumler tablosundaki bolum_id'ye bağlı
    -- Yani ogrenciler.bolum_id değeri, bolumler tablosunda var olmak ZORUNDA
    FOREIGN KEY (bolum_id) REFERENCES bolumler(bolum_id)
);
```

Örnek 5: CHECK kısıtlaması ile tablo oluşturma

```
-- Dersler tablosu
-- CHECK: Belirli bir koşulu sağlamayan veri girişini engeller
CREATE TABLE dersler (
    ders_id INT PRIMARY KEY AUTO_INCREMENT,
    ders_adi VARCHAR(100) NOT NULL,
    ders_kodu VARCHAR(10) NOT NULL UNIQUE,  -- 'YBS101' gibi — benzersiz olmalı
    kredi INT NOT NULL,
    akts INT NOT NULL,
    bolum_id INT,
    -- CHECK: Kredi 1 ile 6 arasında olmalı, aksi halde veri girişi reddedilir
    CHECK (kredi BETWEEN 1 AND 6),
    -- CHECK: AKTS 1 ile 30 arasında olmalı
    CHECK (akts BETWEEN 1 AND 30),
    FOREIGN KEY (bolum_id) REFERENCES bolumler(bolum_id)
);
```

Örnek 6: Çoktan çoğa ilişki tablosu (Junction Table)

```
-- Notlar tablosu: Hangi öğrenci hangi dersten kaç almış
-- Bu tablo ogrenciler ve dersler arasında köprü görevi görür
-- Bir öğrenci birden fazla ders alabilir, bir dersi birden fazla öğrenci alabili
```

```
CREATE TABLE notlar (
    not_id INT PRIMARY KEY AUTO_INCREMENT,
    ogrenci_id INT NOT NULL,
    ders_id INT NOT NULL,
    vize_notu DECIMAL(5,2),                -- 85.50 gibi değerler alabilir
    final_notu DECIMAL(5,2),
    harf_notu CHAR(2),                     -- 'AA', 'BA', 'BB', 'FF' gibi
    donem VARCHAR(20) NOT NULL,            -- '2024-2025 Güz' gibi
    -- Her öğrenci aynı dersi aynı dönemde sadece 1 kez alabilir
    UNIQUE (ogrenci_id, ders_id, donem),
    -- Notlar 0-100 arasında olmalı
    CHECK (vize_notu >= 0 AND vize_notu <= 100),
    CHECK (final_notu >= 0 AND final_notu <= 100),
    FOREIGN KEY (ogrenci_id) REFERENCES ogrenciler(ogrenci_id),
    FOREIGN KEY (ders_id) REFERENCES dersler(ders_id)
);
```

Örnek 7: CREATE TABLE ... AS SELECT (Varolan veriden tablo oluşturma)

```
-- Mevcut bir sorgu sonucundan yeni tablo oluştur
-- Bu komut hem tabloyu oluşturur hem veriyi kopyalar
-- Yapı + veri birlikte gelir, ama constraint'ler (PK, FK) GELMEZ
CREATE TABLE ybs_ogrencileri AS
SELECT ogrenci_id, ad, soyad, email
FROM ogrenciler
WHERE bolum_id = 1;
```

1.4 ALTER TABLE — Tablo Yapısını Değiştirme

Mantık: Tablo zaten oluşturulduktan sonra, yapısında değişiklik yapmak istersen ALTER TABLE kullanırsın. Yeni sütun eklemek, sütun silmek, sütun tipini değiştirmek, constraint eklemek/kaldırmak — hepsi ALTER TABLE ile yapılır.

Örnek 1: Yeni sütun ekleme (ADD)

```
-- Öğrenciler tablosuna telefon numarası sütunu ekle
-- Mevcut satırlarda bu sütun NULL olarak başlar
ALTER TABLE ogrenciler
ADD telefon VARCHAR(15);


-- Birden fazla sütun ekleme
```

```
ALTER TABLE ogrenciler
ADD adres TEXT,
ADD sehir VARCHAR(50) DEFAULT 'Gümüşhane';
```

Örnek 2: Sütun silme (DROP COLUMN)

```
-- Artık ihtiyaç duyulmayan sütunu sil
-- DİKKAT: Sütundaki tüm veriler de gider!
ALTER TABLE ogrenciler
DROP COLUMN adres;
```

Örnek 3: Sütunun veri tipini değiştirme (MODIFY / ALTER COLUMN)

```
-- MySQL söz dizimi: MODIFY kullanılır
-- telefon sütununun uzunluğunu 15'ten 20'ye çıkar
ALTER TABLE ogrenciler
MODIFY COLUMN telefon VARCHAR(20);


-- SQL Server söz dizimi: ALTER COLUMN kullanılır
ALTER TABLE ogrenciler
ALTER COLUMN telefon VARCHAR(20);
```

Örnek 4: Sütun adını değiştirme (RENAME COLUMN)

```
-- MySQL'de sütun adını değiştirme
ALTER TABLE ogrenciler
RENAME COLUMN telefon TO cep_telefonu;
```

Örnek 5: Constraint ekleme

```
-- Mevcut tabloya PRIMARY KEY ekleme
-- Tablo oluşturulurken PK belirtilmediyse sonradan eklenebilir
ALTER TABLE bolumler
ADD PRIMARY KEY (bolum_id);


-- FOREIGN KEY ekleme
-- ogretim_gorevlileri tablosundaki bolum_id'yi bolumler tablosuna bağla
ALTER TABLE ogretim_gorevlileri
ADD FOREIGN KEY (bolum_id) REFERENCES bolumler(bolum_id);
```

```
-- CHECK constraint ekleme
-- Cinsiyet sadece 'E' veya 'K' olabilsin
ALTER TABLE ogrenciler
ADD CHECK (cinsiyet IN ('E', 'K'));


-- UNIQUE constraint ekleme
ALTER TABLE dersler
ADD UNIQUE (ders_kodu);
```

Örnek 6: Constraint kaldırma

```
-- MySQL'de index/constraint kaldırma
ALTER TABLE ogrenciler
DROP INDEX email;

-- Foreign key kaldırma (constraint adı gerekir)
ALTER TABLE ogrenciler
DROP FOREIGN KEY fk_bolum;
```

Örnek 7: DEFAULT değer ekleme/kaldırma

```
-- Varsayılan değer ekleme
ALTER TABLE ogrenciler
ALTER COLUMN sehir SET DEFAULT 'Belirtilmedi';

-- Varsayılan değeri kaldırma
ALTER TABLE ogrenciler
ALTER COLUMN sehir DROP DEFAULT;
```

Örnek 8: Tablo adını değiştirme

```
-- Tablo adını değiştir
ALTER TABLE ogretim_gorevlileri
RENAME TO hocalar;
```

1.5 DROP TABLE — Tablo Silme

Mantık: Tabloyu yapısıyla, verileriyle, index'leriyle birlikte tamamen siler. Geri dönüşü yoktur.

```
-- Tabloyu tamamen sil
DROP TABLE notlar;


-- Tablo yoksa hata verme, sessizce geç
DROP TABLE IF EXISTS notlar;
```

DROP vs DELETE farkı:

- `DROP TABLE` : Tablonun kendisini siler (yapı + veri + her şey)
- `DELETE` : Sadece tablodaki verileri siler, tablo yapısı durur

1.6 TRUNCATE TABLE — Tablo Verilerini Temizleme

Mantık: Tablodaki TÜM verileri siler ama tablonun yapısı (sütunlar, constraint'ler) aynen kalır. DELETE'den farkı: çok daha hızlıdır çünkü satır satır silmez, tüm veriyi bir seferde atar. WHERE koşulu kullanılamaz.

```
-- Notlar tablosundaki tüm verileri sil, ama tablo yapısı kalsın
-- Yeni dönem başında eski notları temizlemek gibi düşün
TRUNCATE TABLE notlar;
```

TRUNCATE vs DELETE vs DROP karşılaştırması:

Özellik	DELETE	TRUNCATE	DROP
Veriyi siler mi?	Evet	Evet	Evet
Yapıyı siler mi?	Hayır	Hayır	Evet
WHERE kullanılabilir mi?	Evet	Hayır	-
Geri alınabilir mi? (ROLLBACK)	Evet	Hayır*	Hayır
Hız	Yavaş	Hızlı	Hızlı
AUTO_INCREMENT sıfırlanır mı?	Hayır	Evet	-

*Bazı veritabanlarında TRUNCATE de ROLLBACK edilebilir (örn. SQL Server transaction içinde).

1.7 CREATE INDEX — İndeks Oluşturma

Mantık: İndeks, veritabanının belirli sütunlardaki verileri daha hızlı bulmasını sağlar. Kitaptaki "dizin/index" sayfası gibi düşün — konuyu tek tek sayfa çevirerek aramak yerine, dizinden bakıp direkt sayfaya gidersin.

Ne zaman index oluşturmalı?

- Sık sık WHERE koşulunda kullanılan sütunlara
- JOIN işlemlerinde kullanılan sütunlara
- ORDER BY ile sıralanan sütunlara

Ne zaman index oluşturmamalı?

- Çok az verisi olan tablolara (gereksiz overhead yaratır)
- Sık INSERT/UPDATE yapılan sütunlara (index'in de güncellenmesi gerektiği için yavaşlatır)

```
-- Öğrencilerin soyadına göre hızlı arama yapabilmek için index oluştur
CREATE INDEX idx_ogrenci_soyad
ON ogrenciler (soyad);


-- Birden fazla sütun üzerinde birleşik index (composite index)
-- Ad + soyad birlikte aranıyorsa performansı artırır
CREATE INDEX idx_ogrenci_ad_soyad
ON ogrenciler (ad, soyad);


-- UNIQUE INDEX: Hem hız sağlar hem de değerlerin benzersiz olmasını garanti eder
CREATE UNIQUE INDEX idx_ders_kodu
ON dersler (ders_kodu);


-- Index silme
DROP INDEX idx_ogrenci_soyad ON ogrenciler;
```

1.8 CREATE VIEW — Görünüm Oluşturma

Mantık: View, sık kullandığın bir sorguyu sanal tablo olarak kaydetmektir. Gerçek veri tutmaz, her çağrıldığında arka plandaki sorguyu çalıştırır. Karmaşık sorguları

basitleştirmek ve güvenlik sağlamak için kullanılır.

```
-- Öğrenci bilgilerini bölüm adıyla birlikte gösteren view
-- Her seferinde JOIN yazmak yerine bu view'ı sorgularsın
CREATE VIEW v_ogrenci_detay AS
SELECT
    o.ogrenci_id,
    o.ad,
    o.soyad,
    o.email,
    b.bolum_adi,
    b.fakulte
FROM ogrenciler o
JOIN bolumler b ON o.bolum_id = b.bolum_id;

-- Kullanımı: Normal tablo gibi sorgulanır
-- SELECT * FROM v_ogrenci_detay;


-- View güncelleme (varsa üzerine yaz)
CREATE OR REPLACE VIEW v_ogrenci_detay AS
SELECT
    o.ogrenci_id,
    o.ad,
    o.soyad,
    o.email,
    o.kayit_tarihi,   -- Yeni sütun ekledik
    b.bolum_adi,
    b.fakulte
FROM ogrenciler o
JOIN bolumler b ON o.bolum_id = b.bolum_id;


-- View silme
DROP VIEW v_ogrenci_detay;
DROP VIEW IF EXISTS v_ogrenci_detay;
```

1.9 CONSTRAINT'LER — Özet ve Karşılaştırma

Constraint	Ne Yapar?	Örnek
NOT NULL	Sütun boş bırakılamaz	`ad VARCHAR(50) NOT NULL`
	Değerler tekrar edemez (NULL	

UNIQUE	**olabilir)**	`email VARCHAR(100) UNIQUE`
PRIMARY KEY	**NOT NULL + UNIQUE birleşimi, tabloda 1 tane olur**	`ogrenci_id INT PRIMARY KEY`
FOREIGN KEY	**Başka tablodaki PK'ya referans**	`FOREIGN KEY (bolum_id) REFERENCES bolumler(bolum_id)`
CHECK	**Belirli bir koşulu zorlar**	`CHECK (kredi BETWEEN 1 AND 6)`
DEFAULT	**Değer verilmezse otomatik atanacak değer**	`sehir VARCHAR(50) DEFAULT 'Gümüşhane'`

BÖLÜM 2: DML (Data Manipulation Language) — Veri İşleme Dili

DML, tablolardaki verileri eklemek, sorgulamak, güncellemek ve silmek için kullanılır. DDL yapıyla ilgilenirken, DML veriyle ilgilenir.

Temel DML komutları: `INSERT` , `SELECT` , `UPDATE` , `DELETE`

2.1 INSERT INTO — Veri Ekleme

Mantık: Tabloya yeni satır (kayıt) ekler. İki temel kullanım şekli var: tüm sütunlara değer vererek veya belirli sütunlara değer vererek.

Örnek 1: Tüm sütunlara değer vererek ekleme

```
-- Sütun adları belirtilmezse, tablodaki sütun sırasına göre değer ver
-- bolumler tablosu: bolum_id, bolum_adi, fakulte, kurulus_tarihi, kontenjan
INSERT INTO bolumler
VALUES (1, 'Yönetim Bilişim Sistemleri', 'İktisadi ve İdari Bilimler', '2010-09-0
```

Örnek 2: Belirli sütunlara değer vererek ekleme (önerilen yöntem)

```
-- Hangi sütuna hangi değerin gittiği açıkça belirtilir
-- Sıra önemli değil, sütun adı-değer eşleşmesi önemli
```

```
-- Belirtilmeyen sütunlar NULL veya DEFAULT değer alır
INSERT INTO bolumler (bolum_id, bolum_adi, fakulte)
VALUES (2, 'Bilgisayar Mühendisliği', 'Mühendislik');
-- kurulus_tarihi = NULL, kontenjan = 60 (DEFAULT) olur
```

Örnek 3: Birden fazla satırı tek seferde ekleme

```
-- Tek INSERT ile birden fazla kayıt eklemek daha performanslıdır
-- Her satır için ayrı INSERT yazmak yerine toplu ekleme yap
INSERT INTO bolumler (bolum_id, bolum_adi, fakulte, kurulus_tarihi, kontenjan)
VALUES
    (3, 'Elektrik-Elektronik Mühendisliği', 'Mühendislik', '2005-09-01', 50),
    (4, 'İşletme', 'İktisadi ve İdari Bilimler', '1998-09-01', 80),
    (5, 'İnşaat Mühendisliği', 'Mühendislik', '2003-09-01', 55);
```

Örnek 4: AUTO_INCREMENT olan tabloya veri ekleme

```
-- ogrenci_id AUTO_INCREMENT olduğu için biz değer vermiyoruz
-- Sistem otomatik olarak 1, 2, 3... şeklinde ID atar
INSERT INTO ogrenciler (ad, soyad, dogum_tarihi, cinsiyet, email, bolum_id)
VALUES ('Emin', 'Alan', '2002-05-15', 'E', 'emin@mail.com', 1);

INSERT INTO ogrenciler (ad, soyad, dogum_tarihi, cinsiyet, email, bolum_id)
VALUES ('Ayşe', 'Yılmaz', '2003-03-20', 'K', 'ayse@mail.com', 1);

INSERT INTO ogrenciler (ad, soyad, dogum_tarihi, cinsiyet, email, bolum_id)
VALUES ('Mehmet', 'Demir', '2001-11-08', 'E', 'mehmet@mail.com', 2);

INSERT INTO ogrenciler (ad, soyad, dogum_tarihi, cinsiyet, email, bolum_id)
VALUES ('Zeynep', 'Kaya', '2002-07-12', 'K', 'zeynep@mail.com', 2);

INSERT INTO ogrenciler (ad, soyad, dogum_tarihi, cinsiyet, email, bolum_id)
VALUES ('Ali', 'Özkan', '2000-01-30', 'E', 'ali@mail.com', 3);
```

Örnek 5: Toplu öğretim görevlisi ekleme

```
INSERT INTO ogretim_gorevlileri (ad, soyad, email, unvan, bolum_id)
VALUES
    ('Ahmet', 'Çelik', 'ahmet.celik@uni.edu.tr', 'Prof. Dr.', 1),
    ('Fatma', 'Arslan', 'fatma.arslan@uni.edu.tr', 'Doç. Dr.', 1),
    ('Hasan', 'Koç', 'hasan.koc@uni.edu.tr', 'Dr. Öğr. Üyesi', 2),
    ('Elif', 'Şahin', 'elif.sahin@uni.edu.tr', 'Öğr. Gör.', 3);
```

Örnek 6: Ders ve not ekleme

```
-- Dersler
INSERT INTO dersler (ders_adi, ders_kodu, kredi, akts, bolum_id)
VALUES
    ('Veritabanı Yönetim Sistemleri', 'YBS201', 3, 5, 1),
    ('Programlama I', 'YBS101', 4, 6, 1),
    ('Veri Yapıları', 'BM201', 3, 5, 2),
    ('Devre Analizi', 'EE101', 4, 7, 3),
    ('İstatistik', 'YBS301', 3, 5, 1);

-- Notlar
INSERT INTO notlar (ogrenci_id, ders_id, vize_notu, final_notu, harf_notu, donem)
VALUES
    (1, 1, 85.00, 90.00, 'AA', '2024-2025 Güz'),
    (1, 2, 70.00, 75.00, 'BB', '2024-2025 Güz'),
    (2, 1, 60.00, 55.00, 'CB', '2024-2025 Güz'),
    (2, 2, 90.00, 95.00, 'AA', '2024-2025 Güz'),
    (3, 3, 45.00, 40.00, 'FF', '2024-2025 Güz'),
    (4, 3, 80.00, 85.00, 'BA', '2024-2025 Güz'),
    (5, 4, 55.00, 60.00, 'CB', '2024-2025 Güz'),
    (1, 5, 92.00, 88.00, 'AA', '2024-2025 Güz');
```

Örnek 7: INSERT INTO ... SELECT (Başka sorgudan veri ekleme)

```
-- Bir sorgunun sonucunu başka tabloya ekle
-- Önce hedef tablo oluşturulmuş olmalı
CREATE TABLE basarili_ogrenciler (
    ogrenci_id INT,
    ad VARCHAR(50),
    soyad VARCHAR(50),
    final_notu DECIMAL(5,2)
);

-- Final notu 80 ve üzeri olan öğrencileri basarili_ogrenciler tablosuna kopyala
INSERT INTO basarili_ogrenciler (ogrenci_id, ad, soyad, final_notu)
SELECT o.ogrenci_id, o.ad, o.soyad, n.final_notu
FROM ogrenciler o
JOIN notlar n ON o.ogrenci_id = n.ogrenci_id
WHERE n.final_notu >= 80;
```

2.2 SELECT — Veri Sorgulama

Mantık: Veritabanından veri çekmek (okumak) için kullanılır. SQL'in en çok kullanılan ve en güçlü komutu. WHERE, ORDER BY, GROUP BY, JOIN gibi pek çok yan cümleyle zenginleştirilir.

Örnek 1: Tüm verileri çekme

```
-- Tablodaki TÜM satırları ve TÜM sütunları getir
-- * (yıldız) "hepsini getir" demek
SELECT * FROM ogrenciler;
```

Örnek 2: Belirli sütunları seçme

```
-- Sadece ad ve soyad sütunlarını getir
-- Gereksiz veri çekmemek performans açısından önemli
SELECT ad, soyad FROM ogrenciler;
```

Örnek 3: DISTINCT — Tekrar eden değerleri eleme

```
-- Hangi bölümlerden öğrenci var? (Tekrar eden bölüm ID'lerini gösterme)
SELECT DISTINCT bolum_id FROM ogrenciler;


-- Hangi şehirlerden öğrenci var?
SELECT DISTINCT sehir FROM ogrenciler;
```

Örnek 4: WHERE — Koşullu sorgulama

```
-- Sadece erkek öğrencileri getir
SELECT * FROM ogrenciler
WHERE cinsiyet = 'E';


-- 2002 ve sonrası doğumlu öğrenciler
SELECT ad, soyad, dogum_tarihi
FROM ogrenciler
WHERE dogum_tarihi >= '2002-01-01';


-- Belirli bir bölümdeki öğrenciler
SELECT ad, soyad
FROM ogrenciler
WHERE bolum_id = 1;
```

Örnek 5: AND, OR, NOT — Çoklu koşul

```
-- YBS bölümünde VE erkek olan öğrenciler
-- AND: İki koşulun da doğru olması gerekir
SELECT ad, soyad
FROM ogrenciler
WHERE bolum_id = 1 AND cinsiyet = 'E';


-- YBS VEYA Bilgisayar Mühendisliği bölümündeki öğrenciler
-- OR: Koşullardan en az birinin doğru olması yeterli
SELECT ad, soyad, bolum_id
FROM ogrenciler
WHERE bolum_id = 1 OR bolum_id = 2;


-- Mühendislik fakültesinde OLMAYAN bölümler
-- NOT: Koşulu tersine çevirir
SELECT bolum_adi
FROM bolumler
WHERE NOT fakulte = 'Mühendislik';


-- Karmaşık koşul: YBS'de olan VE (erkek VEYA 2002 sonrası doğumlu) öğrenciler
-- Parantez kullanarak öncelik belirle
SELECT ad, soyad, cinsiyet, dogum_tarihi
FROM ogrenciler
WHERE bolum_id = 1 AND (cinsiyet = 'E' OR dogum_tarihi > '2002-01-01');
```

Örnek 6: ORDER BY — Sıralama

```
-- Öğrencileri soyadına göre A'dan Z'ye sırala (varsayılan ASC = artan)
SELECT ad, soyad
FROM ogrenciler
ORDER BY soyad;


-- Final notuna göre yüksekten düşüğe sırala (DESC = azalan)
SELECT o.ad, o.soyad, n.final_notu
FROM ogrenciler o
JOIN notlar n ON o.ogrenci_id = n.ogrenci_id
ORDER BY n.final_notu DESC;


-- Önce bölüme göre artan, sonra ada göre artan sırala
-- İki seviyeli sıralama
```

```
SELECT ad, soyad, bolum_id
FROM ogrenciler
ORDER BY bolum_id ASC, ad ASC;
```

Örnek 7: SELECT TOP / LIMIT — Satır sayısını sınırlama

```
-- MySQL'de: İlk 3 öğrenciyi getir
SELECT * FROM ogrenciler
LIMIT 3;


-- SQL Server'da: İlk 3 öğrenciyi getir
SELECT TOP 3 * FROM ogrenciler;


-- En yüksek 3 final notunu getir
SELECT o.ad, o.soyad, n.final_notu
FROM ogrenciler o
JOIN notlar n ON o.ogrenci_id = n.ogrenci_id
ORDER BY n.final_notu DESC
LIMIT 3;
```

Örnek 8: LIKE — Desen (pattern) eşleme

```
-- Adı 'A' ile başlayan öğrenciler
-- % : Sıfır veya daha fazla herhangi bir karakter anlamına gelir
SELECT * FROM ogrenciler
WHERE ad LIKE 'A%';


-- Soyadı 'an' ile biten öğrenciler
SELECT * FROM ogrenciler
WHERE soyad LIKE '%an';


-- Email içinde 'mail' geçen öğrenciler
SELECT * FROM ogrenciler
WHERE email LIKE '%mail%';


-- Adı tam 4 harften oluşan öğrenciler
-- _ (alt çizgi): Tam olarak 1 karakter anlamına gelir
SELECT * FROM ogrenciler
WHERE ad LIKE '____';
```

```
-- Adının ikinci harfi 'e' olan öğrenciler
SELECT * FROM ogrenciler
WHERE ad LIKE '_e%';
```

Örnek 9: IN — Bir listedeki değerlerle eşleme

```
-- Bölüm ID'si 1, 2 veya 3 olan öğrenciler
-- IN, birden fazla OR yazmak yerine kullanılır
SELECT * FROM ogrenciler
WHERE bolum_id IN (1, 2, 3);
-- Şuna eşdeğer: WHERE bolum_id = 1 OR bolum_id = 2 OR bolum_id = 3


-- Harf notu AA veya BA olan notlar
SELECT * FROM notlar
WHERE harf_notu IN ('AA', 'BA');
```

Örnek 10: BETWEEN — Aralık sorgusu

```
-- Vize notu 60 ile 80 arasında olan kayıtlar (60 ve 80 dahil)
SELECT * FROM notlar
WHERE vize_notu BETWEEN 60 AND 80;
-- Şuna eşdeğer: WHERE vize_notu >= 60 AND vize_notu <= 80


-- 2002 ile 2003 arasında doğan öğrenciler
SELECT ad, soyad, dogum_tarihi
FROM ogrenciler
WHERE dogum_tarihi BETWEEN '2002-01-01' AND '2003-12-31';
```

Örnek 11: IS NULL / IS NOT NULL — Boş değer kontrolü

```
-- Email adresi girilmemiş öğrenciler
-- NULL kontrolü için = değil, IS NULL kullanılır!
-- NULL = NULL ifadesi FALSE döner, bu SQL'in özel bir kuralıdır
SELECT * FROM ogrenciler
WHERE email IS NULL;


-- Final notu girilmiş (boş olmayan) kayıtlar
SELECT * FROM notlar
WHERE final_notu IS NOT NULL;
```

Örnek 12: Aliases (AS) — Takma ad verme

```
-- Sütunlara takma ad verme — sonuç tablosunda bu isim görünür
SELECT
    ad AS ogrenci_adi,
    soyad AS ogrenci_soyadi,
    email AS eposta
FROM ogrenciler;


-- Tablolara kısa takma ad verme — özellikle JOIN'lerde işe yarar
-- 'o' yazmak 'ogrenciler' yazmaktan çok daha pratik
SELECT o.ad, o.soyad, b.bolum_adi
FROM ogrenciler AS o
JOIN bolumler AS b ON o.bolum_id = b.bolum_id;
```

Örnek 13: Aggregate Functions — Toplam fonksiyonlar

```
-- COUNT: Kaç kayıt var?
-- Toplam öğrenci sayısı
SELECT COUNT(*) AS toplam_ogrenci FROM ogrenciler;


-- COUNT belirli bir sütunda: NULL olmayanları sayar
SELECT COUNT(email) AS email_girilmis FROM ogrenciler;


-- SUM: Toplam
-- Tüm derslerin toplam kredisi
SELECT SUM(kredi) AS toplam_kredi FROM dersler;


-- AVG: Ortalama
-- Tüm vize notlarının ortalaması
SELECT AVG(vize_notu) AS vize_ortalamasi FROM notlar;


-- MIN ve MAX: En küçük ve en büyük değer
SELECT
    MIN(final_notu) AS en_dusuk_final,
    MAX(final_notu) AS en_yuksek_final
FROM notlar;
```

Örnek 14: GROUP BY — Gruplama

```
-- Her bölümde kaç öğrenci var?
-- GROUP BY: Belirli bir sütuna göre satırları gruplar
-- Aggregate fonksiyonlarla birlikte kullanılır
SELECT bolum_id, COUNT(*) AS ogrenci_sayisi
FROM ogrenciler
GROUP BY bolum_id;


-- Her dersin vize ortalaması
SELECT ders_id, AVG(vize_notu) AS vize_ortalamasi
FROM notlar
GROUP BY ders_id;


-- Her harf notundan kaç tane var?
SELECT harf_notu, COUNT(*) AS sayi
FROM notlar
GROUP BY harf_notu
ORDER BY sayi DESC;
```

Örnek 15: HAVING — Grupları filtreleme

```
-- HAVING: GROUP BY sonrasında grupları filtrelemek için kullanılır
-- WHERE satırları filtreler, HAVING grupları filtreler
-- 2'den fazla öğrencisi olan bölümler
SELECT bolum_id, COUNT(*) AS ogrenci_sayisi
FROM ogrenciler
GROUP BY bolum_id
HAVING COUNT(*) > 2;


-- Vize ortalaması 70'in üzerinde olan dersler
SELECT ders_id, AVG(vize_notu) AS ort
FROM notlar
GROUP BY ders_id
HAVING AVG(vize_notu) > 70;
```

WHERE vs HAVING farkı:

- `WHERE` : GROUP BY'dan ÖNCE çalışır, tekil satırları filtreler
- `HAVING` : GROUP BY'dan SONRA çalışır, grupları filtreler

Örnek 16: JOIN — Tabloları birleştirme

```sql
-- INNER JOIN: İki tabloda da eşleşen kayıtları getirir
-- Sadece bölümü olan öğrenciler görünür, bölümsüz öğrenci veya öğrencisiz bölüm
SELECT o.ad, o.soyad, b.bolum_adi
FROM ogrenciler o
INNER JOIN bolumler b ON o.bolum_id = b.bolum_id;


-- LEFT JOIN: Sol tablodaki (ogrenciler) TÜM kayıtları getirir
-- Eşleşme yoksa sağ taraftaki sütunlar NULL olur
-- Bölümü olmayan öğrenciler de görünür (bolum_adi = NULL)
SELECT o.ad, o.soyad, b.bolum_adi
FROM ogrenciler o
LEFT JOIN bolumler b ON o.bolum_id = b.bolum_id;


-- RIGHT JOIN: Sağ tablodaki (bolumler) TÜM kayıtları getirir
-- Öğrencisi olmayan bölümler de görünür (ad, soyad = NULL)
SELECT o.ad, o.soyad, b.bolum_adi
FROM ogrenciler o
RIGHT JOIN bolumler b ON o.bolum_id = b.bolum_id;


-- FULL OUTER JOIN: Her iki tablodaki TÜM kayıtları getirir
-- Eşleşme olmayanlar NULL ile doldurulur
-- MySQL'de doğrudan desteklenmez, UNION ile yapılır
SELECT o.ad, o.soyad, b.bolum_adi
FROM ogrenciler o
LEFT JOIN bolumler b ON o.bolum_id = b.bolum_id
UNION
SELECT o.ad, o.soyad, b.bolum_adi
FROM ogrenciler o
RIGHT JOIN bolumler b ON o.bolum_id = b.bolum_id;


-- Çoklu JOIN: 3 tabloyu birleştirme
-- "Hangi öğrenci hangi dersten kaç almış" sorusunun cevabı
SELECT
    o.ad,
    o.soyad,
    d.ders_adi,
    d.ders_kodu,
    n.vize_notu,
    n.final_notu,
    n.harf_notu
FROM notlar n
JOIN ogrenciler o ON n.ogrenci_id = o.ogrenci_id
JOIN dersler d ON n.ders_id = d.ders_id
```

```
ORDER BY o.soyad, d.ders_adi;


-- 4 tablo birleştirme: Öğrenci + Bölüm + Ders + Not
SELECT
    o.ad || ' ' || o.soyad AS ogrenci,  -- MySQL'de CONCAT(o.ad, ' ', o.soyad)
    b.bolum_adi,
    d.ders_adi,
    n.harf_notu
FROM notlar n
JOIN ogrenciler o ON n.ogrenci_id = o.ogrenci_id
JOIN dersler d ON n.ders_id = d.ders_id
JOIN bolumler b ON o.bolum_id = b.bolum_id;
```

Örnek 17: UNION — Sorgu sonuçlarını birleştirme

```
-- UNION: İki sorgunun sonuçlarını alt alta birleştirir
-- Tekrar eden satırları eler (DISTINCT gibi)
-- Sütun sayısı ve tipleri aynı olmalı!
SELECT ad, soyad FROM ogrenciler WHERE bolum_id = 1
UNION
SELECT ad, soyad FROM ogrenciler WHERE cinsiyet = 'K';


-- UNION ALL: Tekrar edenleri elemez, hepsini gösterir
-- UNION'dan daha hızlıdır çünkü tekrar kontrolü yapmaz
SELECT ad, soyad FROM ogrenciler WHERE bolum_id = 1
UNION ALL
SELECT ad, soyad FROM ogrenciler WHERE cinsiyet = 'K';
```

Örnek 18: Subquery (Alt sorgu)

```
-- Alt sorgu: Bir sorgunun içinde başka bir sorgu
-- En yüksek final notunu alan öğrenciyi bul
SELECT ad, soyad
FROM ogrenciler
WHERE ogrenci_id = (
    SELECT ogrenci_id
    FROM notlar
    ORDER BY final_notu DESC
    LIMIT 1
);


-- Vize ortalamasının üzerinde not alan öğrenciler
```

```
SELECT o.ad, o.soyad, n.vize_notu
FROM ogrenciler o
JOIN notlar n ON o.ogrenci_id = n.ogrenci_id
WHERE n.vize_notu > (SELECT AVG(vize_notu) FROM notlar);


-- IN ile alt sorgu: Hiç ders almamış öğrenciler
SELECT ad, soyad
FROM ogrenciler
WHERE ogrenci_id NOT IN (SELECT DISTINCT ogrenci_id FROM notlar);
```

Örnek 19: EXISTS

```
-- EXISTS: Alt sorgu en az 1 satır döndürüyorsa TRUE döner
-- En az bir dersten notu olan öğrenciler
SELECT ad, soyad
FROM ogrenciler o
WHERE EXISTS (
    SELECT 1 FROM notlar n
    WHERE n.ogrenci_id = o.ogrenci_id
);
```

Örnek 20: CASE — Koşullu ifade

```
-- CASE: SQL'deki if-else yapısı
-- Notlara göre durum belirle
SELECT
    o.ad,
    o.soyad,
    n.final_notu,
    CASE
        WHEN n.final_notu >= 90 THEN 'Pekiyi'
        WHEN n.final_notu >= 80 THEN 'İyi'
        WHEN n.final_notu >= 70 THEN 'Orta'
        WHEN n.final_notu >= 60 THEN 'Geçer'
        ELSE 'Kaldı'
    END AS durum
FROM ogrenciler o
JOIN notlar n ON o.ogrenci_id = n.ogrenci_id;
```

2.3 UPDATE — Veri Güncelleme

Mantık: Tablodaki mevcut verileri değiştirir. WHERE koşulu MUTLAKA yazılmalı, yoksa tablodaki TÜM satırlar güncellenir!

Örnek 1: Tek satır güncelleme

```
-- Emin'in email adresini güncelle
UPDATE ogrenciler
SET email = 'emin.alan@uni.edu.tr'
WHERE ogrenci_id = 1;
```

Örnek 2: Birden fazla sütunu güncelleme

```
-- Tek UPDATE ile birden fazla sütunu değiştir
UPDATE ogrenciler
SET
    email = 'ayse.yilmaz@uni.edu.tr',
    sehir = 'İstanbul'
WHERE ogrenci_id = 2;
```

Örnek 3: Koşullu toplu güncelleme

```
-- Final notu 50'nin altında olan herkesin harf notunu FF yap
-- WHERE olmadan yazsan TÜM notları FF yaparsın — büyük hata!
UPDATE notlar
SET harf_notu = 'FF'
WHERE final_notu < 50;
```

Örnek 4: Hesaplama ile güncelleme

```
-- Tüm öğrencilerin vize notlarına 5 puan ekle (isteğe bağlı artırım)
-- Ama 100'ü geçmesin
UPDATE notlar
SET vize_notu = CASE
    WHEN vize_notu + 5 > 100 THEN 100
    ELSE vize_notu + 5
END;
```

Örnek 5: Alt sorgu ile güncelleme

```
-- YBS bölümündeki öğrencilerin şehrini 'Gümüşhane' yap
UPDATE ogrenciler
```

```
SET sehir = 'Gümüşhane'
WHERE bolum_id = (SELECT bolum_id FROM bolumler WHERE bolum_adi = 'Yönetim Bilişi
```

Örnek 6: JOIN ile güncelleme (MySQL)

```
-- Mühendislik fakültesindeki tüm bölümlerin kontenjanını 70 yap
UPDATE bolumler
SET kontenjan = 70
WHERE fakulte = 'Mühendislik';
```

Örnek 7: WHERE olmadan UPDATE — TEHLİKELİ!

```
-- DİKKAT: Bu komut tablodaki HER satırı günceller!
-- Sınavda bu hatayı yapmamak için WHERE'i kontrol et
UPDATE ogrenciler
SET sehir = 'Bilinmiyor';
-- Tüm öğrencilerin şehri 'Bilinmiyor' olur!
```

2.4 DELETE — Veri Silme

Mantık: Tablodan satır siler. UPDATE gibi, WHERE koşulu MUTLAKA yazılmalı. WHERE olmadan tablodaki TÜM satırlar silinir!

Örnek 1: Tek satır silme

```
-- ID'si 5 olan öğrenciyi sil
DELETE FROM ogrenciler
WHERE ogrenci_id = 5;
```

Örnek 2: Koşullu silme

```
-- FF notu alan tüm kayıtları sil
DELETE FROM notlar
WHERE harf_notu = 'FF';
```

Örnek 3: Çoklu koşullu silme

```
-- 2024-2025 Güz döneminde VE final notu 30'un altında olan notları sil
```

```
DELETE FROM notlar
WHERE donem = '2024-2025 Güz' AND final_notu < 30;
```

Örnek 4: Alt sorgu ile silme

```
-- Bilgisayar Mühendisliği bölümündeki öğrencilerin notlarını sil
DELETE FROM notlar
WHERE ogrenci_id IN (
    SELECT ogrenci_id FROM ogrenciler WHERE bolum_id = 2
);
```

Örnek 5: Tüm verileri silme (WHERE'siz DELETE)

```
-- Tablodaki TÜM satırları siler, ama tablo yapısı kalır
-- TRUNCATE'ten farkı: Daha yavaştır ama ROLLBACK yapılabilir
DELETE FROM notlar;
```

Örnek 6: FOREIGN KEY olan tabloda silme sorunu

```
-- Bu hata verir! Çünkü bolumler tablosundaki bolum_id'ye
-- ogrenciler tablosundan referans var (FOREIGN KEY)
-- Önce bağımlı tabloları temizlemen lazım
DELETE FROM bolumler WHERE bolum_id = 1;
-- HATA: Cannot delete or update a parent row: a foreign key constraint fails


-- Doğru sıra: Önce bağımlı kayıtları sil, sonra ana kaydı
DELETE FROM notlar WHERE ogrenci_id IN
    (SELECT ogrenci_id FROM ogrenciler WHERE bolum_id = 1);
DELETE FROM ogrenciler WHERE bolum_id = 1;
DELETE FROM dersler WHERE bolum_id = 1;
DELETE FROM bolumler WHERE bolum_id = 1;
```

BÖLÜM 3: SINAV İÇİN KRİTİK KARŞILAŞTIRMALAR

DELETE vs TRUNCATE vs DROP

```
DELETE FROM notlar;
-- Satır satır siler, log tutar, ROLLBACK yapılabilir
-- WHERE ile koşullu silme yapılabilir
-- AUTO_INCREMENT sıfırlanmaz

TRUNCATE TABLE notlar;
-- Tüm veriyi bir seferde atar, çok hızlı
-- WHERE kullanılamaz
-- AUTO_INCREMENT sıfırlanır

DROP TABLE notlar;
-- Tabloyu tamamen yok eder (yapı + veri + index + constraint)
-- Geri dönüş yok
```

DDL vs DML Özet

DDL (Yapı İşlemleri)	DML (Veri İşlemleri)
CREATE TABLE	INSERT INTO
ALTER TABLE	SELECT
DROP TABLE	UPDATE
TRUNCATE TABLE	DELETE
CREATE INDEX	
CREATE VIEW	

SQL Sorgu Çalışma Sırası (Execution Order)

Yazma sırası ile çalışma sırası FARKLIDIR! Sınavda bunu bilmek çok önemli:

```
Çalışma sırası:
1. FROM / JOIN    → Hangi tablodan?
2. WHERE          → Satır filtresi
3. GROUP BY       → Gruplama
4. HAVING         → Grup filtresi
5. SELECT         → Hangi sütunlar?
6. DISTINCT       → Tekrarları ele
7. ORDER BY       → Sıralama
8. LIMIT / TOP    → Satır sayısı sınırlama
```

Wildcard Karakterler

Karakter	Anlamı	Örnek
%	Sıfır veya daha fazla karakter	'A%' → A ile başlayan
_	Tam 1 karakter	'_e%' → İkinci harfi e olan

BÖLÜM 4: SINAVDA ÇIKABİLECEK ÖRNEK SORULAR

Soru 1: "Mühendislik fakültesindeki bölümlerin ders sayısını ve ortalama kredisini bulunuz."

```
SELECT b.bolum_adi, COUNT(*) AS ders_sayisi, AVG(d.kredi) AS ort_kredi
FROM dersler d
JOIN bolumler b ON d.bolum_id = b.bolum_id
WHERE b.fakulte = 'Mühendislik'
GROUP BY b.bolum_adi;
```

Soru 2: "Her öğrencinin aldığı derslerin ağırlıklı not ortalamasını hesaplayınız."

```
SELECT
    o.ad,
    o.soyad,
    SUM(n.final_notu * d.kredi) / SUM(d.kredi) AS agirlikli_ortalama
FROM notlar n
JOIN ogrenciler o ON n.ogrenci_id = o.ogrenci_id
JOIN dersler d ON n.ders_id = d.ders_id
GROUP BY o.ogrenci_id, o.ad, o.soyad;
```

Soru 3: "Hiç ders almamış öğrencileri listeleyin."

```
-- Yöntem 1: NOT IN ile
SELECT ad, soyad
FROM ogrenciler
WHERE ogrenci_id NOT IN (SELECT DISTINCT ogrenci_id FROM notlar);

-- Yöntem 2: LEFT JOIN + IS NULL ile
SELECT o.ad, o.soyad
```

```
FROM ogrenciler o
LEFT JOIN notlar n ON o.ogrenci_id = n.ogrenci_id
WHERE n.not_id IS NULL;

-- Yöntem 3: NOT EXISTS ile
SELECT o.ad, o.soyad
FROM ogrenciler o
WHERE NOT EXISTS (
    SELECT 1 FROM notlar n WHERE n.ogrenci_id = o.ogrenci_id
);
```

Soru 4: "Öğrenci sayısı 2'den fazla olan bölümleri, öğrenci sayısına göre azalan sırayla listeleyin."

```
SELECT b.bolum_adi, COUNT(o.ogrenci_id) AS ogrenci_sayisi
FROM bolumler b
JOIN ogrenciler o ON b.bolum_id = o.bolum_id
GROUP BY b.bolum_adi
HAVING COUNT(o.ogrenci_id) > 2
ORDER BY ogrenci_sayisi DESC;
```

Soru 5: "Dersler tablosuna 'donem' sütunu ekleyiniz ve varsayılan değeri 'Güz' olsun."

```
ALTER TABLE dersler
ADD donem VARCHAR(10) DEFAULT 'Güz';
```

Bu rehber vize sınavındaki DDL ve DML konularının tamamını kapsar. Her komutu mantığıyla birlikte, aynı senaryo üzerinden, birden fazla kullanım şekliyle öğrendin.